

Содержание

Контрольные вопросы к лабораторным работам	1
Лабораторная №1 — Базовые навыки работы с одноплатным компьютером	1
Лабораторная №2 — Работа с платой по SSH	2
Лабораторная №3 — Кросс-компилятор	4
Лабораторная №4 — Ядро Linux	5
Лабораторная №5 — Компоненты образа	6
Лабораторная №6 — Запись образа на носитель	8
Лабораторная №7 — SPI-интерфейс	9
Лабораторная №8 — I2C-интерфейс	10
Лабораторная №9 — Логические анализаторы и GPIO	12
Лабораторная №10 — Основы программирования микроконтроллеров Arduino	13
Лабораторная №11 — Интеграция Arduino и Lichee RV Dock по SPI с датчиком BME280	14
Лабораторная №12 — Протокол MQTT	16
Лабораторная №13 — Итоговое проектное задание по вариантам	19

Контрольные вопросы к лабораторным работам

Лабораторная №1 — Базовые навыки работы с одноплатным компьютером

1. Чем открытая архитектура набора команд RISC-V отличается от x86-64?

Ответ: RISC-V имеет открытую модульную спецификацию: небольшая базовая ISA дополняется стандартными расширениями, а собственную реализацию процессора можно создавать без лицензирования самой ISA. x86-64 развивает совместимый набор команд семейства x86, контролируется правообладателями и реализуется производителями по лицензионным соглашениям. В контексте курса Allwinner D1 использует 64-разрядное ядро RISC-V XuanTie C906.

2. Почему для первого подключения к одноплатнику используется UART, а не SSH?

Ответ: UART работает на аппаратном уровне через последовательный порт и не требует настройки сети, IP-адреса и работающего SSH-сервера. Сразу после подачи питания на плату UART выдаёт логи загрузчика и ядра, а затем приглашение консоли. SSH требует предварительной настройки сети и демона `sshd` — это невозможно сделать при первом включении, пока система ещё не сконфигурирована.

3. Что такое GPIO и какую роль играет распиновка (pinout) при подключении периферии?

Ответ: GPIO (General Purpose Input/Output) — линии ввода-вывода общего назначения, через которые процессор взаимодействует с внешними устройствами. Распиновка (pinout) — это схема соответствия физических контактов на разъёме платы конкретным GPIO-линиям процессора. Без неё невозможно определить, к какому выводу подключать датчик или периферию, и как этот вывод сконфигурирован (основная функция или альтернативная — UART, SPI, I2C).

4. Как определить, к какому файлу устройства в /dev/ подключился UART-конвертер?

Ответ: Сразу после подключения преобразователя выполните `sudo dmesg | grep -i tty`: ядро выведет строку с именем последовательного устройства, например `ttyUSB0`. Также можно сравнить вывод `ls -la /dev/ttyUSB*` до и после подключения.

5. Что показывает команда `lscpu` на одноплатнике и как интерпретировать поле Architecture для RISC-V?

Ответ: Команда `lscpu` выводит архитектуру процессора, количество ядер, тактовую частоту, порядок байт (endianness) и другие характеристики CPU. Для Allwinner D1 в поле Architecture будет указано `riscv64`, что означает 64-битную архитектуру RISC-V. Это подтверждает, что система работает на RISC-V процессоре, а не на `x86_64` или ARM.

Лабораторная №2 — Работа с платой по SSH

1. В чём принципиальное отличие SSH от UART-соединения?

Ответ: UART — прямое аппаратное соединение через последовательный порт, работающее на физическом уровне без шифрования. SSH — сетевой протокол прикладного уровня, работающий поверх TCP/IP и обеспечивающий шифрование трафика, аутентификацию и защищённую передачу данных. SSH позволяет подключаться удалённо

через сеть (Ethernet/Wi-Fi); UART требует физического кабеля USB-UART и не даёт выхода в сеть.

2. Как безопасно проверить изменение `/etc/openssh/sshd_config` и убедиться, что сервер принимает соединения?

Ответ: Перед перезапуском нужно выполнить `sshd -t`: при корректной конфигурации команда завершается без сообщения, а при ошибке указывает проблемную строку. Затем выполняют `systemctl restart sshd`, проверяют `systemctl status sshd --no-pager` и слушающий порт командой `ss -lntp 'sport = :22'`. До проверки нового входа нельзя закрывать резервную UART-сессию. Для обычной работы используют отдельного пользователя и ключи; включать `PermitRootLogin yes` не требуется.

3. Зачем нужна утилита `scp` и как она работает?

Ответ: `scp` (Secure Copy) копирует файлы между хостами по защищённому SSH-соединению. Она использует тот же механизм аутентификации и шифрования, что и SSH, поэтому отдельно настраивать доступ не требуется. Пример: `scp file.txt root@192.168.1.100:/home/` копирует локальный файл на удалённую машину. `scp` удобна для передачи скомпилированных программ, конфигурационных файлов и Device Tree на одноплатник.

4. Как настроить Wi-Fi-соединение на одноплатнике без графического интерфейса?

Ответ: С помощью утилиты `nmcli` (NetworkManager command line). Последовательность действий: `nmcli device wifi list` — просмотр доступных сетей; `nmcli device wifi connect "SSID" password "пароль"` — подключение. Проверить статус можно командой `nmcli connection show` или `ip a`. После настройки соединение сохраняется и автоматически поднимается при загрузке.

5. Почему рекомендуется отключать аутентификацию по паролю и переходить на ключи SSH?

Ответ: Аутентификация по ключам SSH криптографически значительно надёжнее парольной: злоумышленник не может подобрать ключ перебором (в отличие от пароля). Закрытый ключ хранится только на клиентской машине, открытый — на сервере; пароль же передаётся (хоть и в зашифрованном виде) и уязвим к перехвату при компрометации сервера. После настройки ключей параметры `PasswordAuthentication no` и `PermitRootLogin prohibit-password` в `sshd_config` закрывают векторы атак.

6. **Что делает команда `ssh-copy-id` и какие файлы ключа допустимо передавать на сервер?**

Ответ: `ssh-copy-id -i ~/.ssh/id_ed25519.pub пользователь@сервер` добавляет содержимое открытого ключа `.pub` в файл `~/.ssh/authorized_keys` удалённого пользователя и выставляет подходящие права доступа. Закрытый ключ `~/.ssh/id_ed25519` остаётся только на клиентском компьютере и никогда не передаётся на сервер. После копирования вход проверяют в отдельной сессии, например с `ssh -o BatchMode=yes`.

Лабораторная №3 — Кросс-компилятор

1. **Что такое кросс-компиляция и в каких случаях она необходима?**

Ответ: Кросс-компиляция — сборка исполняемого файла на одной архитектуре (host, например x86-64) для выполнения на другой (target, например RISC-V). Она необходима, когда целевая платформа недостаточно производительна для компиляции или не содержит инструментов разработки. В лабораторной №3 тестовая программа собирается для RISC-V на рабочей станции x86-64; в последующих работах используются также нативная компиляция на Lichee и интерпретируемые Python-программы.

2. **Зачем нужен флаг `-static` при линковке программ для RISC-V?**

Ответ: Флаг `-static` компоует все необходимые библиотеки (включая `libc`) внутрь исполняемого файла, делая его полностью автономным. Это критически важно, если на целевом одноплатнике отсутствуют или несовместимы динамические библиотеки для RISC-V. Статическая линковка также упрощает перенос программы — достаточно скопировать один файл, не заботясь о зависимостях.

3. **Какую роль выполняет `qemu-user-static` и `binfmt_misc` в процессе кросс-компиляции?**

Ответ: `qemu-user-static` — эмулятор пользовательского режима, способный выполнять бинарные файлы чужой архитектуры (например, RISC-V на x86_64). `binfmt_misc` — механизм ядра Linux, позволяющий регистрировать обработчики для исполняемых файлов по сигнатуре. Вместе они позволяют запускать RISC-V программы на x86_64-хосте прозрачно, как нативные, без эмуляции целой системы — просто `./program` работает.

4. Чем отличается тулчейн `gcc` от `riscv64-linux-gnu-gcc`?

Ответ: `gcc` компилирует программы для архитектуры хоста (`x86_64`), в то время как `riscv64-linux-gnu-gcc` — кросс-компилятор, генерирующий машинный код для архитектуры RISC-V (64-бит). Второй использует другой ассемблер, компоновщик и стандартную библиотеку — `glibc`, собранную под RISC-V. На уровне исходного кода программа та же; разница — в целевом наборе инструкций, который генерирует компилятор.

Лабораторная №4 — Ядро Linux

1. Какие основные функции выполняет ядро Linux в системе?

Ответ: Ядро управляет аппаратными ресурсами (процессор, память, устройства ввода-вывода), предоставляет процессы и планировщик, обеспечивает изоляцию между процессами, реализует файловые системы и сетевой стек, а также предоставляет интерфейс системных вызовов для взаимодействия пользовательских программ с оборудованием. Без ядра невозможна многозадачность, работа с файлами и сетью.

2. Для чего используется `make menuconfig` при сборке ядра?

Ответ: `make menuconfig` запускает текстовый интерфейс (на базе `ncurses`) для конфигурации ядра, где можно включать и отключать драйверы, модули, файловые системы, сетевые протоколы и другие опции. На выходе генерируется файл `.config`, который определяет, какие именно компоненты будут скомпилированы в ядро. Это позволяет собрать минимальное ядро, содержащее только нужные для конкретной платы драйверы.

3. Почему при сборке ядра необходимо указывать `ARCH=riscv` и `CROSS_COMPILE=riscv64-linux-gnu-`?

Ответ: `ARCH=riscv` сообщает системе сборки, для какой архитектуры процессора собирается ядро — от этого зависят специфичные для RISC-V исходные файлы, заголовки и сборочные правила. `CROSS_COMPILE=riscv64-linux-gnu-` задаёт префикс кросс-компилятора — утилиты `make` будут вызывать `riscv64-linux-gnu-gcc` вместо системного `gcc`. Без этих переменных ядро соберётся под архитектуру хоста (`x86_64`), а не под RISC-V.

4. Что такое `PREEMPT_RT` и в каких сценариях он необходим?

Ответ: PREEMPT_RT (Real-Time) — патч к ядру Linux, превращающий его в операционную систему реального времени с гарантированным временем отклика на события. Он делает планировщик полностью вытесняющим даже в критических секциях ядра. Необходим в системах, где критична детерминированная задержка обработки: управление двигателями, промышленная автоматизация, сбор данных с АЦП с жёсткими временными ограничениями.

5. Как правильно выбрать целевой SoC (Allwinner D1) в menuconfig?

Ответ: Воспользоваться встроенным поиском: нажать /, ввести sun20i или D1, найти опцию ARCH_SUNXI (поддержка SoC Allwinner sunxi) и убедиться, что она включена. Дополнительно в разделе Device Drivers для Allwinner D1 необходимо активировать драйверы периферии (SPI_SUN6I, DMA_SUN6I, I2C_MV64XXX и др.), специфичные для sun20i. Правильный выбор SoC гарантирует, что в ядро попадут нужные драйверы и таблицы совместимости.

Лабораторная №5 — Компоненты образа

1. Что такое Device Tree и какую роль он выполняет при загрузке Linux?

Ответ: Device Tree (DT) — структура данных, описывающая аппаратную конфигурацию платы: какие устройства присутствуют, на каких адресах, к каким GPIO и прерываниям подключены. Загрузчик (U-Boot) передаёт указатель на DT ядру Linux, и ядро на основе этой информации загружает соответствующие драйверы и конфигурирует периферию. Без DT ядро не знает, какое оборудование установлено на плате.

2. Чем отличается файл .dts от .dtb?

Ответ: .dts (Device Tree Source) — текстовый файл, написанный человеком и читаемый в редакторе, описывающий аппаратную конфигурацию. .dtb (Device Tree Blob) — скомпилированная бинарная версия .dts, которую загрузчик передаёт ядру. Компиляция выполняется утилитой dtc (Device Tree Compiler). В репозитории ядра .dts-файлы хранятся в arch/riscv/boot/dts/allwinner/.

3. Для чего нужны SPL и U-Boot proper в цепочке загрузки?

Ответ: U-Boot инициализирует оперативную память (DDR), загружает ядро Linux и Device Tree, и передаёт управление ядру. На Allwinner

D1 загрузка проходит в два этапа: SPL (Secondary Program Loader) — минимальный код, помещающийся во внутреннюю SRAM и инициализирующий DDR; затем U-Boot Proper — полноценный загрузчик, загружающий ядро с SD-карты или сети.

4. Какую роль выполняет OpenSBI в цепочке загрузки RISC-V?

Ответ: OpenSBI (Open Supervisor Binary Interface) — это реализация firmware-уровня для RISC-V, работающая в M-mode (Machine mode — самый привилегированный). Она предоставляет стандартизованный интерфейс (ecall) для вышестоящего ПО (ядра Linux), через который ядро выполняет привилегированные операции: установку таймеров, межпроцессорные прерывания, перезагрузку. OpenSBI загружается до ядра и остаётся резидентным в памяти.

5. Какая информация содержится в файле extlinux.conf?

Ответ: extlinux.conf — конфигурационный файл для загрузчика U-Boot в режиме sysboot/extlinux. Он содержит метку загрузочной записи (label), путь к ядру (kernel /Image), путь к DTB (fdt /sun20i-d1-lichee-rv-dock.dtb) и параметры командной строки ядра (append root=LABEL=R00TFS rootwait console=ttyS0,115200). Файл позволяет менять параметры загрузки без перекомпиляции U-Boot.

6. Почему путь root=LABEL=R00TFS устойчивее имени вида /dev/mmcblk0p2?

Ответ: Имя /dev/mmcblk0p2 зависит от порядка обнаружения контроллеров и носителей ядром, поэтому на другой конфигурации номер устройства может измениться. Метка файловой системы R00TFS хранится на самом разделе и позволяет ядру найти нужный rootfs независимо от присвоенного имени устройства. Метка должна быть уникальной и совпадать со значением, заданным при форматировании.

7. Почему file и ненулевой размер не заменяют SHA-256 и проверку содержимого?

Ответ: file определяет формат по отдельным сигнатурам, а ненулевой размер подтверждает только наличие данных. Усечённый или изменённый файл может сохранить сигнатуру и по-прежнему распознаваться корректно. SHA-256 обнаруживает любое изменение относительно заранее зафиксированного эталона, а специализированные проверки, например xz -t, tar -tJf и декомпиляция DTB, подтверждают внутреннюю целостность и ожидаемое содержимое.

8. Для чего при воспроизводимой сборке используется SOURCE_DATE_EPOCH

Ответ: U-Boot по умолчанию встраивает текущие дату и время сборки, из-за чего два образа из одинаковых исходников могут получить разные байты и SHA-256. SOURCE_DATE_EPOCH задаёт фиксированную временную метку, обычно время выбранного commit. При одинаковых исходниках, инструментах, OpenSBI и параметрах это устраняет один из источников невоспроизводимости.

Лабораторная №6 — Запись образа на носитель

1. Почему для загрузочной SD-карты необходимо создавать два раздела?

Ответ: В этой работе используется схема BOOT/VFAT + ROOTFS/ext4: первый раздел содержит ядро, Device Tree и extlinux.conf, а второй — корневую файловую систему Linux. Такое разделение удобно, но не является единственно возможным: U-Boot может читать и другие поддерживаемые файловые системы, включая ext4.

2. Зачем загрузчик U-Boot записывается на SD-карту со смещением 8 килобайт (bs=1k seek=8)?

Ответ: Смещение 8 КиБ задано схемой загрузки Allwinner: BootROM ищет SPL/U-Boot в предусмотренной области носителя. MBR занимает первый сектор, U-Boot начинается с байта 8192, а первый раздел в этой работе — с сектора 2048. Команда: `dd if=u-boot-sunxi-with-spl.bin of=/dev/sdX bs=1K seek=8`.

3. Какая файловая система используется для раздела BOOT и почему?

Ответ: FAT32 (vfat). Она выбрана потому, что U-Boot имеет встроенную поддержку чтения FAT и может загрузить ядро и Device Tree напрямую из FAT32-раздела. FAT32 проста, не требует сложных драйверов в загрузчике и поддерживается всеми операционными системами, что удобно при обновлении файлов ядра и dtb с хост-компьютера.

4. Какие компоненты необходимо скопировать на SD-карту для успешной загрузки системы?

Ответ: На SD-карту необходимо записать: (1) U-Boot SPL+Proper по смещению 8 КБ (`dd seek=8`); (2) на раздел BOOT (FAT32): ядро Image, Device Tree Blob .dtb, файл extlinux.conf; (3) на раздел ROOTFS (ext4): корневую файловую систему (распакованный rootfs tarball

ALT Linux или другой дистрибутив для RISC-V). Без любого из этих компонентов загрузка не состоится.

Лабораторная №7 — SPI-интерфейс

1. **Какие четыре сигнала используются в интерфейсе SPI и за что отвечает каждый?**

Ответ: MOSI (Master Out Slave In) — данные от ведущего к ведомому; MISO (Master In Slave Out) — данные от ведомого к ведущему; SCK (Serial Clock) — тактовый сигнал, генерируемый ведущим; $\sim$ CS (Chip Select, он же SS) — выбор ведомого устройства активным низким уровнем. Все четыре линии обязательны; MOSI и MISO обеспечивают полнодуплексный обмен.

2. **Чем архитектура «ведущий-ведомый» в SPI отличается от шинной архитектуры I2C?**

Ответ: В SPI каждому ведомому устройству требуется отдельная линия $\sim$ CS, что ограничивает масштабирование, но обеспечивает полнодуплекс и высокую скорость. В I2C все устройства делят две общие линии (SDA/SCL), а выбор устройства осуществляется по адресу, передаваемому в начале транзакции. SPI — протокол «точка-точка» с индивидуальным выбором; I2C — многоточечная шина с адресацией.

3. **Почему в Device Tree статус интерфейса (status) по умолчанию имеет значение "disabled"?**

Ответ: Включающие файлы (.dtsi) описывают все аппаратные возможности SoC, но конкретная плата использует лишь часть из них. Статус "disabled" означает, что устройство описано, но не активно — оно не будет инициализировано ядром. Активация производится в файле конкретной платы (.dts) переопределением: status = "okay". Это предотвращает конфликты из-за неиспользуемых или конфликтующих выводов.

4. **Зачем нужен драйвер SPIDEV и какое символьное устройство появляется в /dev/ после его активации?**

Ответ: SPIDEV — драйвер ядра, предоставляющий доступ к SPI-устройствам из пользовательского пространства через символьные файлы /dev/spidevX.Y. Без него программы на Python (через библиотеку spidev) не могут взаимодействовать с SPI-интерфейсом. После активации в /dev/ появляется файл вида /dev/spidev1.0, где 1 — номер шины, 0 — номер устройства на шине.

5. **Какой идентификатор в поле `compatible` используется для привязки драйвера `spidev` к устройству в Device Tree?**

Ответ: Используется `compatible = "rohm,dh2228fv"`. Этот идентификатор занесён в таблицу совместимости драйвера `spidev` в ядре, поэтому при обнаружении устройства с таким `compatible` ядро автоматически привяжет к нему драйвер `spidev`. Альтернативный подход — добавить новый идентификатор в исходный код драйвера, но использование существующего проще и не требует модификации ядра.

6. **Почему для работы SPI-интерфейса на Allwinner D1 требуется драйвер `DMA_SUN6I`?**

Ответ: Драйвер `SPI_SUN6I` использует каналы прямого доступа к памяти (DMA) для передачи данных между SPI-контроллером и оперативной памятью без участия процессора. Без драйвера `DMA_SUN6I` инициализация SPI-контроллера завершается ошибкой — ядро не может выделить DMA-каналы. DMA также разгружает CPU при высокоскоростных передачах, что критично для SPI на частотах в десятки мегагерц.

Примечание: вопросы по разработке и отладке приложений для RISC-V (hasher, VS Code Remote-SSH, QEMU_LD_PREFIX) вынесены в справочный материал `Additional/remote_development.md`.

Лабораторная №8 — I2C-интерфейс

1. **Почему интерфейс I2C на Lichee RV Dock обозначен как TWI? В чём разница?**

Ответ: TWI (Two-Wire Interface) — промышленное название I2C, используемое Atmel и другими производителями по лицензионным соглашениям (I2C — торговая марка Philips/NXP). Электрически и протоколно TWI идентичен I2C: две линии (SDA/SCL), 7-битная адресация, master-slave. На распиновке Lichee нужно искать пины `TWI2_SDA` и `TWI2_SCL`, а не I2C.

2. **Как организована адресация устройств на шине I2C?**

Ответ: Каждое ведомое устройство на шине имеет уникальный 7-битный адрес (возможна и 10-битная адресация). Ведущий начинает транзакцию с отправки адреса и бита направления (чтение/запись).

Все ведомые слушают шину; отвечает только то устройство, чей адрес совпал. Адрес обычно жёстко задан производителем микросхемы, иногда его можно изменить перемычками на плате (например, BME280: 0x76 или 0x77 в зависимости от уровня на выводе SDO).

3. Чем отличаются линии SDA и SCL по своему назначению в I2C?

Ответ: SDA (Serial Data) — двунаправленная линия данных: ведущий и ведомый поочерёдно выставляют на неё биты данных и подтверждения (ACK). SCL (Serial Clock) — линия тактирования, управляемая исключительно ведущим; она задаёт моменты, когда данные на SDA должны быть стабильны (высокий уровень SCL) и когда могут меняться (низкий уровень SCL). Обе линии используют открытый сток с подтяжкой к питанию.

4. Какие параметры ядра необходимо включить для поддержки I2C на Allwinner D1?

Ответ: Необходимо включить четыре параметра: CONFIG_I2C (базовая поддержка), CONFIG_I2C_MUX (поддержка мультиплексоров шины), CONFIG_I2C_MV64XXX (драйвер контроллера Marvell mv64xxx, совместимый с sunxi-контроллерами) и CONFIG_I2C_CHARDEV (доступ к I2C-устройствам из пользовательского пространства через /dev/i2c-*). В menuconfig они ищутся по этим же строкам ввода через /.

5. Для чего нужен драйвер I2C_CHARDEV и какое устройство появляется в /dev/?

Ответ: I2C_CHARDEV предоставляет символьные файлы /dev/i2c-N для доступа к I2C-адаптерам из userspace. Через эти устройства библиотеки вроде luma-oled отправляют команды дисплею, читают данные датчиков и сканируют шину. Без этого драйвера интерфейс I2C доступен только из кода ядра (других драйверов), но не из Python-скриптов.

6. Как определить I2C-адрес подключённого устройства, если он неизвестен?

Ответ: Сначала определяют адаптер командой `i2cdetect -l`, затем проверяют разрешённый документацией адрес или согласованный диапазон. Стандартная таблица `i2cdetect -y N` обычно охватывает адреса 0x03–0x77, поскольку часть адресов зарезервирована. SSD-1306 обычно откликается на 0x3C, но фактический адрес следует сверять с документацией модуля.

Лабораторная №9 — Логические анализаторы и GPIO

1. Для чего нужен логический анализатор при отладке цифровых сигналов?

Ответ: Логический анализатор захватывает состояния цифровых линий (лог. 0/1) с высокой частотой дискретизации и отображает их в виде временных диаграмм. В отличие от осциллографа, он показывает не форму сигнала, а его логический смысл: декодирует протоколы (UART, SPI, I2C), измеряет временные интервалы между событиями и выявляет расхождения между ожидаемым и реальным поведением шины. Это основной инструмент отладки цифровых интерфейсов.

2. Какие параметры необходимо настроить в PulseView для декодирования UART-сигнала?

Ответ: После добавления декодера UART нужно задать: номер канала (к какому входу анализатора подключён сигнал), скорость передачи (baud rate — например, 115200), формат кадра (обычно 8N1: 8 бит данных, без контроля чётности, 1 стоп-бит), и порядок бит (LSB first для UART). Частота дискретизации должна быть минимум в 4 раза выше baud rate.

3. Что такое GPIO-линия в терминах `libgpiod2` и как к ней обратиться из программы?

Ответ: В `libgpiod 2.x` GPIO-линия идентифицируется символьным устройством `gpiochipN` и смещением внутри контроллера. Последовательность работы включает `gpiod_chip_open()`, создание настроек через `gpiod_line_settings_new()`, добавление их вызовом `gpiod_line_config_add_line_settings()` и запрос линий через `gpiod_chip_request_lines()`.

4. Чем управление GPIO через `libgpiod2` принципиально отличается от прямой записи в регистры микроконтроллера (как в Arduino)?

Ответ: В Arduino программа может напрямую изменять регистры портов ATmega328P. `libgpiod 2.x` выполняет `gpiod_line_request_set_value()` через GPIO character-device API ядра (`/dev/gpiochipN`), поэтому операция проходит через системный вызов, драйвер и планировщик. Это медленнее прямой записи в регистр, но обеспечивает управление ресурсами, защиту от конфликтов и единый API.

Лабораторная №10 — Основы программирования микроконтроллеров Arduino

1. В чём ключевое отличие программирования микроконтроллера (**bare-metal**) от разработки под операционную систему?

Ответ: На микроконтроллере программа — единственный исполняемый код (firmware), работающий напрямую с регистрами периферии без промежуточных слоёв абстракции. Нет планировщика, нет разделения памяти между процессами, нет драйверов. На ОС (Linux) программа работает в пользовательском пространстве, обращаясь к оборудованию через системные вызовы и драйверы. Bare-metal даёт детерминированное время выполнения и полный контроль над железом ценой отсутствия многозадачности и защиты памяти.

2. Какую роль выполняет загрузчик (**bootloader**) в Arduino?

Ответ: Bootloader — небольшая программа, постоянно хранящаяся в защищённой области flash-памяти ATmega328P. При включении Arduino он проверяет, не поступает ли команда на загрузку новой прошивки по UART. Если команды нет — передаёт управление основной программе. Если есть — принимает новую прошивку, проверяет её целостность и записывает во flash. Это позволяет программировать Arduino через USB без внешнего программатора.

3. Какие режимы срабатывания аппаратных прерываний доступны на ATmega328P и чем они отличаются?

Ответ: Четыре режима: LOW — срабатывает постоянно, пока уровень на пине низкий; CHANGE — срабатывает при любом изменении сигнала (с HIGH на LOW и наоборот); RISING — только при переходе с LOW на HIGH (восходящий фронт); FALLING — только при переходе с HIGH на LOW (нисходящий фронт). Выбор режима зависит от задачи: CHANGE полезен для обнаружения любого фронта (меандр), FALLING/RISING — для реакции на нажатие кнопки.

4. Почему переменные, используемые и в обработчике прерывания, и в основном цикле, должны объявляться с ключевым словом **volatile**?

Ответ: Компилятор может оптимизировать доступ к переменным, кэшируя их в регистрах процессора. Если переменная изменяется в ISR (обработчике прерывания), а основной цикл читает её из регистра, он не увидит изменений. **volatile** запрещает такую оптимизацию: каждое обращение к переменной генерирует фактическое чтение из

памяти. Без `volatile` код может работать некорректно из-за рассинхронизации между `ISR` и `loop()`.

5. Зачем нужен преобразователь логических уровней при соединении Arduino (5V) с Lichee RV Dock (3.3V)?

Ответ: Arduino Uno работает от 5 В и выставляет логическую единицу напряжением около 5 В. GPIO-пины Allwinner D1 на Lichee рассчитаны на 3.3 В — подача 5 В может вывести их из строя. Преобразователь уровней на MOSFET-транзисторах согласует напряжения: сигнал 5 В со стороны Arduino преобразуется в 3.3 В на стороне Lichee, и наоборот. Без преобразователя прямое соединение недопустимо.

6. Почему в обработчике прерывания не рекомендуется использовать функции `delay()` и `Serial.print()`?

Ответ: `delay()` зависит от таймерных прерываний и внутри `ISR` может не завершиться; `Serial.print()` также может заблокироваться, поскольку передача UART использует прерывания. Обработчик должен выполняться максимально быстро, только фиксируя событие и обновляя флаги.

Лабораторная №11 — Интеграция Arduino и Lichee RV Dock по SPI с датчиком BME280

1. Почему для подключения BME280 к Arduino используется I2C, а не SPI?

Ответ: I2C требует всего двух проводов (SDA/SCL) и не занимает дополнительных пинов — это удобно, когда на Arduino уже задействованы четыре пина под SPI-slave (MISO/MOSI/SCK/SS). Кроме того, у Arduino Uno аппаратные пины I2C (A4/A5) и SPI (11–13) разнесены и не конфликтуют. SPI на BME280 занял бы ещё три пина, что привело бы к нехватке выводов.

2. Какую роль выполняет сигнал SS в SPI и почему на Arduino он настроен как `INPUT_PULLUP`?

Ответ: SS (Slave Select) переводит слейв в активный режим: когда уровень LOW — слейв слушает SCK и участвует в обмене; когда HIGH — игнорирует шину. `INPUT_PULLUP` подтягивает линию к HIGH внутренним резистором (~50 кОм), чтобы при отключённом или молчащем мастере слейв гарантированно был неактивен и не реагировал на шум. Мастер (Lichee) опускает SS в LOW через конвертер, преодолевая подтяжку.

3. **Зачем в скетче делается предзагрузка `SPDR = g_tx[0]` в `setup()` до начала каких-либо SPI-транзакций?**

Ответ: Когда мастер начинает самую первую транзакцию ($SS\downarrow$ и тактирование), shift register слейва немедленно начинает выдвигать данные. Если в SPDR ничего не было записано заранее, слейв отправит случайный байт (содержимое неинициализированного регистра). Предзагрузка `SPDR = g_tx[0]` гарантирует, что первый же байт, полученный мастером, будет осмысленным (первый байт температуры).

4. **Почему для передачи 12 байт используется 12 отдельных однобайтовых транзакций, а не одна 12-байтовая? Что произойдёт при burst-передаче?**

Ответ: При burst-передаче следующий байт должен быть записан в SPDR до начала его тактирования; иначе мастер получит устаревшие или повторные данные. Отдельные однобайтовые транзакции в этой работе дают ISR гарантированное время на подготовку следующего байта.

5. **Для чего нужен логический преобразователь уровней? Какие сигналы проходят через него в направлении HV→LV, а какие — LV→HV?**

Ответ: Преобразователь согласует напряжения: Arduino (5V) не может напрямую соединяться с Lichee (3.3V). В направлении HV→LV (5V→3.3V) идёт сигнал **MISO** — Arduino (5V) передаёт данные, Lichee (3.3V) принимает. В направлении LV→HV (3.3V→5V) идут сигналы **MOSI, SCK, SS** — Lichee (3.3V, мастер) управляет шиной, Arduino (5V) принимает. Без преобразователя 5V на входе Lichee могут повредить GPIO-пины.

6. **Как `struct.unpack("<fff", data)` интерпретирует 12 байт, полученных по SPI? Что означает префикс `<`?**

Ответ: Формат "`<fff`" означает: `<` — little-endian (младший байт по младшему адресу, соответствует порядку байт ATmega328P и Allwinner D1), `f` — 4-байтовое число с плавающей точкой одинарной точности (IEEE 754 float32). Три `f` последовательно извлекают из 12 байт три числа: температуру, влажность и давление. Каждое занимает 4 байта. Без префикса `<` по умолчанию использовался бы native-порядок, который на некоторых архитектурах — big-endian, что привело бы к неверным значениям.

7. **Какие регистры ATmega328P управляют работой SPI в режиме слейва? Какие биты необходимо установить?**

Ответ: Основной регистр — **SPCR** (SPI Control Register). Для включения слейва с прерываниями необходимо установить биты: **SPE** (SPI Enable, бит 6) — включает модуль SPI; **SPIE** (SPI Interrupt Enable, бит 7) — разрешает прерывание по завершению байта. Бит MSTR (4) оставляется сброшенным (0 = slave). Дополнительно: **SPSR** (Status Register) содержит бит SPIF (7), сигнализирующий о завершении передачи; **SPDR** — буфер данных, через который слейв получает и отправляет байты.

8. Как вызовы `spi.xfer2()` образуют отдельные однобайтовые транзакции?

Ответ: `xfer2()` удерживает CS активным между элементами одного вызова и освобождает его после завершения вызова. Поэтому каждый отдельный `xfer2([0x00])` образует завершённую однобайтовую транзакцию; между вызовами Arduino получает время подготовить следующий байт в SPDR.

9. Как проверить корректность SPI-обмена с помощью логического анализатора? Какие сигналы должны быть видны на каждом канале?

Ответ: Подключить каналы анализатора к пинам Lichee: D0=CS, D1=SCK, D2=MOSI, D3=MISO. В PulseView настроить декодер SPI: CPOL=0, CPHA=0, MSB first, 8 бит, CS active-low. На исправной системе будет видно 12 коротких импульсов CS↓, каждый сопровождается 8 тактами SCK (период ~1 мкс при 1 МГц). MOSI во всех битах = 0 (Lichee шлёт пустышки). MISO показывает биты данных датчика, меняющиеся от байта к байту (в отличие от MOSI).

10. Почему важно соединить все GND (Arduino, Lichee, конвертер) в одну общую точку? Что произойдёт, если этого не сделать?

Ответ: Все цифровые сигналы измеряются относительно общего уровня «земли» (0V). Если GND устройств не соединены, их «земли» могут иметь разный потенциал (плавать), и логические уровни (0/1) не будут корректно распознаваться принимающей стороной. Например, 3.3V на Lichee относительно своего GND может выглядеть как 1.5V или 5V относительно GND конвертера — результат непредсказуем. Без общего GND обмен данными через SPI невозможен.

Лабораторная №12 — Протокол MQTT

1. В чём преимущества архитектуры «издатель-подписчик» по сравнению с клиент-серверной для IoT-систем?

Ответ: В архитектуре «издатель-подписчик» (Pub/Sub) издатель не знает о получателях — он публикует данные в топик, а брокер доставляет их всем подписавшимся. Это устраняет жёсткую связь между компонентами: можно добавлять новых подписчиков (веб-интерфейс, база данных, система оповещения) без изменения кода издателя. Клиент-серверная архитектура требует, чтобы клиент знал адрес сервера, а сервер — адрес клиента, что плохо масштабируется при десятках и сотнях устройств.

2. Какие уровни QoS существуют в MQTT и в каких сценариях каждый из них целесообразно использовать?

Ответ: QoS 0 («at most once») — сообщение отправляется один раз без подтверждения; подходит для телеметрии, где потеря нескольких измерений не критична (температура раз в минуту). QoS 1 («at least once») — брокер подтверждает получение (PUBACK), возможны дубликаты; подходит для команд управления, где важна доставка, а повторная команда безопасна. QoS 2 («exactly once») — четырёхэтапное рукопожатие гарантирует ровно одну доставку; подходит для транзакций, где дубликат недопустим (списание средств).

3. Как обеспечивается безопасность в MQTT? Какие механизмы аутентификации и шифрования поддерживаются?

Ответ: Безопасность обеспечивается на двух уровнях. Транспортный: TLS/SSL шифрует канал между клиентом и брокером (порт 8883), предотвращая перехват и подмену данных. Прикладной: брокер может требовать имя пользователя и пароль (в поле CONNECT-пакета), а также ограничивать доступ к топикам через ACL (Access Control Lists). В production-системах рекомендуется комбинация TLS + аутентификация + ACL.

4. Чем отличается MQTT от других протоколов IoT, таких как CoAP или HTTP/2?

Ответ: MQTT оптимизирован под асимметричные каналы и нестабильные соединения: минимальный заголовок (2 байта), keep-alive, сохранённые сессии, will-сообщения. CoAP (Constrained Application Protocol) основан на REST-модели и UDP — легче для сенсорных сетей, но не имеет встроенной Pub/Sub-маршрутизации. HTTP/2 — тяжёлый протокол с текстовыми заголовками, требующий постоянного TCP-соединения; его накладные расходы избыточны для устройств с килобайтами памяти.

5. Как организовать иерархию топиков для системы умного дома?

Ответ: Иерархия строится через `/`: дом/этаж/комната/устройство/измерение. Примеры: `home/floor1/livingroom/temperature`, `home/floor1/kitchen/humidity`, `home/floor2/bedroom1/light/status`, `home/floor1/livingroom/light/command`. Такая структура позволяет подписываться с wildcard: `home/floor1/+ /temperature` — все датчики температуры на первом этаже; `home/#` — все устройства в доме.

6. Какие проблемы могут возникнуть при использовании MQTT в сетях с нестабильным соединением и как их решить?

Ответ: Основная проблема — разрыв TCP-соединения, при котором брокер не получает keep-alive пакеты и считает клиента отключённым. Решения: (1) QoS 1/2 для гарантированной доставки после переподключения; (2) механизм «сохранённых сессий» (`clean session = false`) — брокер хранит недоставленные сообщения и подписки; (3) Will Message — брокер автоматически публикует заданное сообщение при обрыве связи клиента, уведомляя систему о потере устройства.

7. Как масштабировать MQTT-инфраструктуру при увеличении количества устройств?

Ответ: Варианты масштабирования: (1) мостовое соединение (bridge) нескольких брокеров Mosquitto — каждый обслуживает свой сегмент сети; (2) кластерные брокеры (EMQX, VerneMQ) с автоматическим шардированием топиков по узлам; (3) иерархическая архитектура — локальные брокеры на каждом этаже/здании передают агрегированные данные на центральный. Ключевой фактор — количество одновременных TCP-соединений; Mosquitto на Lichee выдерживает сотни клиентов.

8. В чём преимущества использования MQTT поверх WebSocket для веб-приложений?

Ответ: WebSocket обеспечивает постоянное двустороннее TCP-соединение между браузером и сервером, идеально подходящее для MQTT. Браузерный JavaScript-клиент (Paho JS, MQTT.js) подключается к брокеру через `ws://` или `wss://`, получает данные в реальном времени без периодического опроса (polling). Это позволяет строить живые дашборды, где графики обновляются мгновенно при поступлении данных с датчиков, без перезагрузки страницы.

9. Как реализовать retained messages и will messages в MQTT и для чего они используются?

Ответ: Retained Message (флаг retain = 1 при публикации) — брокер сохраняет последнее сообщение в топике и автоматически доставляет его каждому новому подписчику. Это полезно для статусов, которые меняются редко: новый клиент сразу получает актуальное значение температуры, не дожидаясь следующего измерения. Will Message задаётся при CONNECT: если клиент отключается нештатно (таймаут keep-alive), брокер публикует will-сообщение в указанный топик, оповещая систему о потере устройства.

10. Какие библиотеки для работы с MQTT существуют для различных платформ?

Ответ: Для Python широко используется Eclipse Paho (paho-mqtt), для C/C++ — libmosquitto и Paho C/C++, для JavaScript/Node.js — MQTT.js, для Java — Eclipse Paho Java, для Go — Eclipse Paho Go или gmqtt, для микроконтроллеров Arduino/ESP — PubSubClient и AsyncMqttClient. Выбор зависит от платформы, доступной памяти и требований к TLS/QoS.

Лабораторная №13 — Итоговое проектное задание по вариантам

1. Какую роль в итоговом проекте играет MQTT-брокер Mosquitto и почему он выбран в качестве транспортного протокола?

Ответ: Mosquitto связывает MQTT-клиенты на Lichee и ПК; Arduino подключается к Lichee по SPI только в вариантах, где она предусмотрена. MQTT выбран из-за небольших накладных расходов и модели Pub/Sub, позволяющей запускать издателей и подписчиков независимо.

2. Почему в заданиях на связку Arduino→Lichee используется SPI, а не UART или I2C? Какие преимущества это даёт?

Ответ: SPI обеспечивает полнодуплексный обмен с тактированием от мастера (Lichee), что даёт полный контроль над скоростью передачи и моментами чтения данных. В отличие от UART, SPI не требует согласования baud rate на обеих сторонах. В отличие от I2C, SPI позволяет жёстко управлять транзакцией через сигнал SS — именно это используется в протоколе из 12 однобайтовых транзакций, где подъём SS между байтами заставляет shift register Arduino перезагружаться из SPDR.

3. Каковы основные архитектурные паттерны, повторяющиеся во всех вариантах итогового задания?

Ответ: Общими являются разделение системы на компоненты, обмен между MQTT-клиентами через брокер на Lichee, обработка отказов и наблюдаемая проверка результата. Arduino и SPI используются только в тех вариантах, где это предусмотрено; некоторые проекты работают с периферией непосредственно на Lichee.

4. Какие меры следует предусмотреть для обеспечения надёжности распределённой системы при возможных отказах компонентов?

Ответ: На уровне MQTT: использовать will-сообщения для обнаружения отключения компонентов; QoS 1 для критичных команд. На уровне GUI: отображать статус подключения (зелёный/красный индикатор) с тайм-аутом, как в вариантах 1 и 2 — если данные не поступали дольше N секунд, показывать «нет данных». На уровне Lichee: проверять наличие /dev/spidev* перед началом SPI-обмена, обрабатывать исключения при недоступности Arduino или датчика.

5. Как организовать тестирование и отладку распределённой системы, когда все компоненты работают на разном оборудовании?

Ответ: Поэтапная стратегия «от простого к сложному»: (1) проверить каждый компонент изолированно — датчик через Serial-монитор Arduino, SPI-приём на Lichee утилитами командной строки, MQTT через mosquitto_pub/sub; (2) проверить каждое звено цепочки отдельно: Arduino→Lichee (SPI), Lichee→ПК (MQTT); (3) использовать логический анализатор (fx2lafw + PulseView) для верификации SPI-сигналов; (4) отлаживать MQTT-поток, подписавшись на топики через mosquitto_sub с отдельного терминала; (5) только после подтверждения всех звеньев запускать полную цепочку.